

Structural Computing and Computer Vision

Jett Vanbrocklin

April 2026

1 Introduction

A problem arises when trying to identify the position of a point in 3-D space through the use of cameras. A single frame perceived by a camera loses information, since it only measures a projection of 3-D space onto the image plane. Given a single frame, this problem is not solvable (using a passive camera that senses natural light). However, given multiple frames or multiple cameras, and the known attitude and location of the camera when an image is taken, one could determine the lost dimensionality due to projection of an arbitrary point.

This laboratory report follows an implementation and experimentation of a **structural computer** that determines the 3-D position of a feature given at least 2 2-D projections and camera poses. Furthermore, the report will describe a basic implementation of computer vision to identify balloons in images and feed found coordinates to the structural computer.

2 Implementation

The structural computer described above in **Section 1** is described in greater detail in [1] (Section 9.2). The following section describes the Linear Least Squares optimization method translated into both MatLab and C++ environments. Both implementations follow the same process, and only differ in data structure and camera lens distortion (due to OpenCV's coordinate system contrasting the MatLab implementation).

The section will further describe the method of extracting balloon coordinates within the image plane from raw images using OpenCV.

2.1 Structural Computer in MatLab

Given an array of measured projections of a feature point and respective attitude matrices and position vectors of the camera in the I frame, the task is to estimate the position of the feature point in 3-D space. These structures depend on the number of frames given N , where N must be greater than 2.

The function then constructs the $\mathbf{H}'$ matrix described in [1] using the projection coordinate and the intrinsic pixel size of the camera.

From H' , H and $-z$ can be extracted. Which, along with the accounted noise covariance matrix R , make up the least squares optimal solution expression for $\hat{X}$, the estimated 3-D position of the feature point. A portion of the product within the $\hat{X}$ expression also expresses the solutions error covariance matrix P_x . As mentioned previously, a more detailed follow through of the derivation can be found in [1].

2.2 Structural Computer in C++

It was previously noted that the C++ implementation of the structural computer follows very closely the MatLab Implementation. For more insight on the broad idea, refer to **Section 2.1**.

While the implementation is similar, it is relevant to note the differences. First, the structure that contains feature points and respective camera pose and position is different across the translations. In C++, the feature point is structured directly with its respective camera positional characteristics. Second, due to the variation in OpenCV's coordinate system and the camera distortion, the intrinsic matrix of the camera is altered slightly.

2.3 Finding Balloons with OpenCV

To test the structural computer described above, an algorithm utilizing OpenCV was developed that could identify balloons of a specified color given an image. I will aim to walk through the process of locating balloons in this section. The image is first blurred to prevent noise from setting off the color thresholds and converted to HSV. To identify color, each pixel is checked to be within an HSV range that is defined for each color. In this case the thresholds are as follows:

	Hue	Saturation	Value
Lower Region Min	0	60	100
Lower Region Max	10	255	255
Upper Region Min	160	60	100
Upper Region Max	179	255	255

Table 1: Red Range Values

	Hue	Saturation	Value
Lower Region Min	90	75	60
Lower Region Max	130	255	255

Table 2: Blue Range Values

To further refine the color mask created, erosion is used to breakdown the weaker regions of color. This prepares the image to be traced for contours, which

are curves that can be connected among regions of similar color. The contours have properties such as area and perimeter that allow for distinguishability.

For each contour, a bounding box and enclosing circle are found using OpenCV functions. These characterizing shapes prepare each contour to be checked for several properties that determine a given contour is enclosing a balloon. **1.** The aspect ratio of the enclosing circle is within the range $[1.15, 1.6]$. **2.** Minimum radius for the enclosing circle is $35px$. **3.** Minimum area of the contour is $4000px^2$. **5.** The polygon approximation (OpenCV Tool) finds more than 6 vertices in the contour. **6.** The **circularity** of the contour is greater than 0.7, where circularity is defined as:

$$Circularity = \frac{4\pi A}{P^2} \quad (1)$$

where A is the contour area, and P is the contour perimeter.

Given that these constraints are met, the algorithm determines that a found contour is in reference to a balloon, and the image plane coordinates are pushed into the vector of relevant projections.

3 Experimentation

3.1 Accuracy of the Structural Computer

The structural computer described in section **2.1** can be tested by inputting a minimum of two non-parallel image frames containing the same feature point, Using a test script in MatLab allows for just that, where the feature point $p = [0, 0, 0.5]^T$ is projected onto the image plane of the quad-camera at two different instances of quad attitude and position. The first instance places the quad at $r_{I1} = [1, 0, 1]^T$ with the pose $e_1 = [0, 0, \pi]^T$. The second instance places the quad at $r_{I2} = [0.7071, 0.7071, 1]^T$ with a pose of $e_2 = [0, 0, 5\pi/4]^T$.

By plugging the image plane coordinates found using a camera simulation model (described thoroughly in [3]), and the camera positions and attitude into the structural computer, the 3-D point can be estimated.

	x_E	y_E	z_E
Trial 1	0.0069	0.0035	0.0084
Trial 2	-0.0007	-0.0077	-0.0013
Trial 3	-0.0065	0.0077	0.0047
Average	-0.0001	0.0012	0.0039

Table 3: Estimated 3-D Points Error

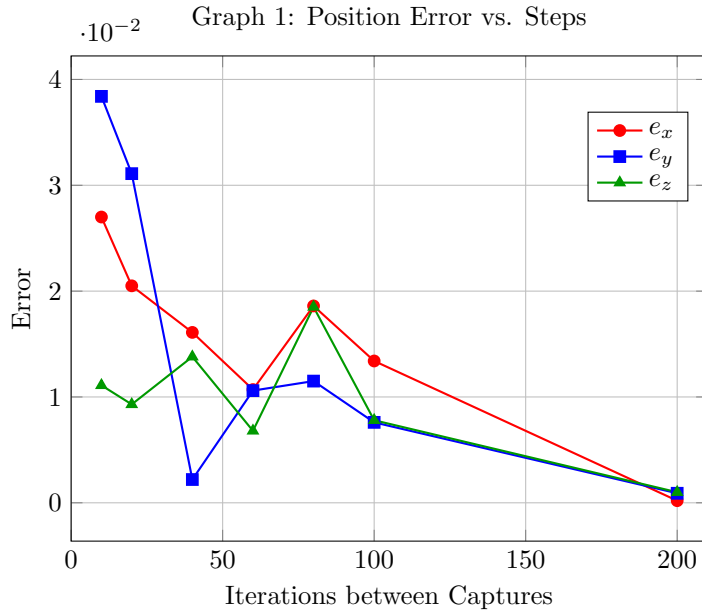
Table 3 above defines the structural computer estimated 3-D point given the previously stated inputs after 3 trials. On average, a 3-D point experiences less than 1 cm of error, and as little as 0.0007 cm of error. The range between

errors is around 1.6 cm, centered at the zero. The accuracy can be defined to the centimeter range, highlighting the effectiveness of this structural computer.

3.2 The Effect of Samples

Using the quadrotor simulated described in detail in [1], [2], and [3], the construction of a feature point's 3-D coordinate can be tested. The circular path of the quad (once again described in [3], **section 4** provides a large stream of quad, and therefore camera positions and attitudes in an environment with a defined virtual feature point resting at $p = [0, 0, 0.7]^T$. By saving the output of the camera simulator, and the respective camera pose and position when capturing the frame, the structural computer can be provided with sufficient data to reconstruct the feature point's 3-D position.

The goal of this experiment is to determine any correlation between the spacing of the captures and the accuracy of the structural computer. The frames used to reconstruct the feature point can be at any interval in its path. In this scenario, the time steps between each capture maintained consistent spacing for each trial.



In **Graph 1** above, the relationship between iterations between captures and error for the x, y, and z component are shown. Where e_x , e_y , and e_z are the respective x, y, and z errors. Based on the trend in the line graph, for all three components of 3-D space, increasing the spacing between captures decreases the

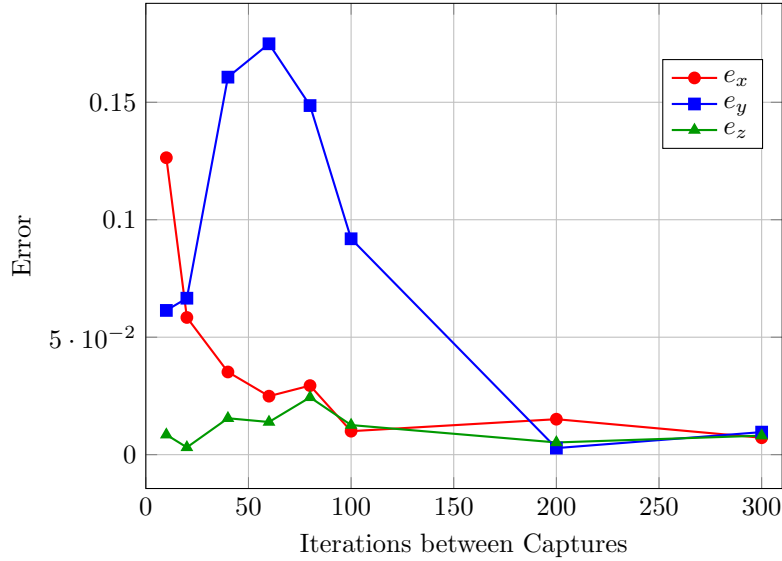
estimation error.

Based on intuition, this data is as expected. If images are taken too close together in a circular path, the risk of having dependent frames, or close to independent, increases. Dependent frames can cause singularities, and may not grant any new information. By spacing apart the captures, the risk of singularities is much lower to near zero. This insures the 10 images captured are able to grant effective information.

3.3 The Effect of Pose Estimation

In the previous section, the relationship between time-steps between captures and accuracy of the structural computer was displayed, portraying a positive correlation between increasing space between captures and accuracy. In that experiment the true state of the quadrotor and the camera were used to compute the location of the feature point. In this section, the same experiment was carried out, but using the estimated state of the quadrotor for each capture.

Graph 2: Position Error vs. Steps under Estimated State



In **Graph 2** above, the relationship between iterations between captures and the error among the three components of position: e_x , e_y , and e_z under an estimated state of the quadrotor and camera is displayed. The trend follows the same pattern from Graph 1 in section 3.2, where increasing the number of iterations between captures causes a convergence of error to 0. There does appear to be a spike in error at around the 50-step range for e_z . Due to the

nature of the spike, it is safe to assume the cause is the increased volatility in error from the added noise the estimation model attempts to simulate.

4 Integration

The C++ structural computer and the balloon finding algorithm described in **section 2** can be pipe-lined to test both the effectiveness of the computer vision color and objection detection and the effectiveness of the structural computer in translating projected coordinates from several frames into 3-D points in space. This section experiments with this pipeline to determine the quality of both algorithms.

4.1 Locating Balloon Position from Image Set

Using a dataset of images that may contain no balloons, a red balloon, a blue balloon, or both, along with their respective camera attitudes and positions, both the balloon finding algorithm and the structural computer were tested.

Firstly, the balloon finding algorithm was tuned to a very high efficiency to ensure that proper data was being directed into the structural computer. The statistics for the balloon-finding rate are as follows: 23 out of 23 fully in frame red balloons found, 27 out of 27 fully in frame blue balloons found, 0 out of 7 partially in frame balloons found.



Figure 1: In Frame Balloon

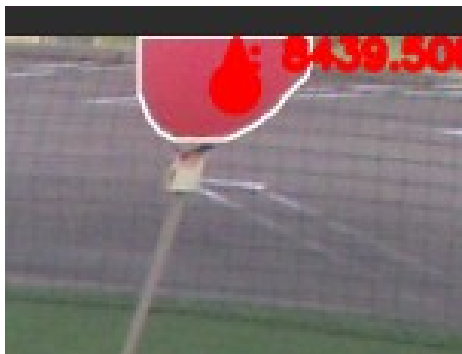


Figure 2: Out of Frame Balloon

Fully in frame balloons (**Figure 1**) can be defined by balloons whose outer contour resides fully within the image. Partially in frame balloons (**Figure 2**) have edges that lie outside of the image frame or behind another object in the foreground. By percentage, 100% of the balloons fully in frame were found and characterized by their color. There were also zero false positives that would skew the 3-D point estimation.

It is also worth noting some relevant annotations in Figure 1. First, the white circle is the calculated center point projection on the image plane. Second, two white lines describe the contour of the features and the minimal enclosing circle of the contour (the contour traces the edge of the balloon). Lastly, the red text was used for debugging purposes, and contains the area of the contour and the aspect ratio of the enclosing circle (though in the figure only the area value is visible).

The calculated center points of the balloon in each image is converted to the image plane coordinate system before being pushed into the r_x vector that contains the feature point projections from each frame. Feeding the found r_x vector with each points respective camera pose yields the following results:

	e_x	e_y	e_z
Blue Balloon	0.00923	0.03464	0.02069
Red Balloon	0.05248	0.0045	0.0105

Table 4: Absolute Value of Estimated 3D Location Errors in Meters

Table 4 above showcases the accuracy of this pipeline, determining the 3-D location of the red and blue balloon with a maximum component of error at 5.25 centimeters.

References

- 1 Humphreys, Todd. "Main." Jan. 2026.
- 2 Vanbrocklin, Jett. "Simulating Quadrotor Dynamics" Feb. 2026.
- 3 Vanbrocklin, Jett. "Simulator Refinements and Quadrotor Control" Feb. 2026.
- 4 Vanbrocklin, Jett. "Measurement Simulation and State Estimation" Mar. 2026.
- 5 Vanbrocklin, Jett. "Path Planning" Mar. 2026
- 6 Humphreys, Todd. "lab5" Mar. 2026.